Understanding Software Engineering through Individual Project

Managing Complexity with Rules and Tools as a Professional Problem Solver

ASE 285

ASE @ Northern Kentucky University

Chapter 1:
Understanding Individual Project

- Tim learns that being a great software engineer means solving problems independently, not just working in teams.
- An individual project lets you choose a problem you care about, design a solution, build it, and test it on your own.
- You set your own goals and deadlines, learning to manage complexity using the rules and tools from class.

Why Work Alone?

Tim

"I thought software engineering was all about teamwork. Why do I need to work alone?"

Julie

"Because in the real world, you need both skills. Sometimes you discover a problem and want to solve it yourself—that's where individual projects matter."

Tim

"Can you give me an example?"

Julie

"Sure! There was a software engineer who was frustrated by slow, storage-limited email systems, he experimented with using web technologies and Google-style search to build something dramatically simpler and more powerful. That small individual project eventually evolved into Gmail and transformed how the world uses email.[1]"

Tim

"Wow! So an individual project can become something big?"

Julie

"Absolutely. You choose a problem you care about, design a solution, build it, and test it yourself. You prove you can solve **any** problem independently."

[1]Gmail began as a 20% individual project by Google engineer Paul Buchheit.

Julie's Story: TravelBuddy

Tim

"What did you build for your individual project?"

Julie

"I love traveling but always struggled with managing itineraries and budgets. So I built 'TravelBuddy'—an app to help organize trips."

Tim

"How did you start? Did you know all the technologies you needed?"

Julie

"Not at all! I knew Node.js, a bit of React, and NoSQL databases. I was in 'unknown unknown' territory—I didn't know what I didn't know."

Tim

"That sounds scary! Weren't you worried?"

Julie

"Yes, but I knew **why** I needed to solve the problem. So I just started with what I knew. Then I discovered Electron and React Native while browsing YouTube—they were exactly what I needed!"

Tim

"That's the Red Car Theory[2] from class, right?"

Julie

"Exactly! I only noticed those tools because I had already started my project."

[2]Red Car Theory: Once you start looking for something specific (like a red car), you suddenly see it everywhere. Your brain filters information until it becomes relevant to you.

Taking Risks

Tim

"But what if you fail? In a team project, that would be a disaster."

Julie

"That's the difference! In team projects, we play it safe to deliver what we promise. In individual projects, we can take risks and learn from mistakes."

Tim

"So individual projects are for learning and exploring?"

Julie

"Yes! Once I knew **why** (solve my travel problem), I figured out **what** (web, desktop, mobile app). The **how** was easy—just use the right tools."

Tim

"So AI and YouTube[3] can teach you 'how,' but you are the only person in the world who can come up with 'why'?"

Julie

"Exactly! And managing complexity was the real part of the project—not the coding[4] I had to revise my plan, switch databases when one didn't work with Electron, and battle endless complexity."

Tim

"But you finished it?"

---

[3]You can find other resources that teach us 'how' to solve the problems, including classroom discussion.

[4]In the real world, most software engineers spend only about 20% of their time for coding/programming. The percentage becomes less and less as they progress their career. Most of their time is used for better problem solving, not coding

Julie

"Yes! I solved the problem I set out to tackle in the end. It wasn't easy, but every obstacle taught me how to manage complexity, and every mistake helped me grow. I learned from my peers, and I was able to support others with the experience I gained. Now I feel confident that I can take on **any** challenge—whether I'm working alone or with a team."

Tim's Turn

Tim thinks about what problem he wants to solve.

> ***Tim thinks***
>
> *"I use ChatGPT every day, but I sometimes cannot find my old prompts and answers. There are too many conversations!"*

He keeps asking.

> ***Tim thinks***
>
> *"What if I could tag my prompts so I can search and organize them easily?"*

Tim smiles. He knows **why** he wants to solve this problem. Now he just needs to figure out **what** to build and **how** to build it.

> **Tim**
>
> "OK. I will call my project TagGPT!"

The journey begins.

Chapter 2:
Features and Requirements

- Tim knows **why** he wants to build TagGPT. Now he needs to decide **what** to build.
- Defining features and requirements for the individual project is exactly the same as for the team project.
- You do not need to learn a different method. Use the same rules and tools that you learned in the team project story.

Same as the Team Project

Tim

"I know **why** I want TagGPT. Now I need to write down the features and requirements. Is it different from the team project?"

Julie

"No. It is exactly the same. There is no reason to make it different."

Tim

"Why not?"

Julie

"Because the goal is the same: turn a problem (**why**) into features and requirements (**what**), and only then decide how to build them (**how**). The size of the team does not change this process."

Tim

"So I should read the features and requirements part of the team project story again?"

Julie

"Yes. Review the team project story, especially how problems become features and how requirements are written. Then apply the same method to your own project."

What Stays the Same

- Start from the problem (**why**), then define features and requirements (**what**).
- Write each requirement in the same format as the team project, and make it testable.
- Create acceptance tests for the requirements.
- Keep the features and requirements in your `docs/` folder and link them from your Canvas page.

Tim

"That makes it easy. I already know the method, so I can focus on my own problem."

Julie

"Exactly. The only difference is that you are the only person who writes and implements them."